SYNTHETIC FINANCIAL DIALOGS · EXTRACTION BENCHMARK

High-level approach for generating household data, dialogs, and extraction metrics

This page summarizes the end-to-end methodology across data generation, dialog generation, extraction scoring, Train/holdout test split construction, and current benchmark results.

SYNTHETIC HOUSEHOLDS

5,000

Generated across 11 explicit scenarios

TRAIN DIALOGS

316

Used to debug and tune the pipeline

HOLDOUT TEST DIALOGS

99

Held out from extractor/evaluator tuning

HOLDOUT TEST MEAN ACCURACY

98.74%

Mean scored field fraction

Overview Data generation Dialog generation Metrics Train and holdout test Results Quality

OVERVIEW

The benchmark is built as a controlled synthetic-data pipeline

Table of contents

1. Synthetic household generation
2. Dialog generation and realism checks

3. Matching, scoring, and errors
4. Train/holdout test split logic
5. Train and holdout test results
6. Quality controls at each stage
7. Next steps

1

Generate households

Sample affluent US RIA-like households from explicit scenario priors, public-data anchors, and lifecycle constraints.

2

Generate dialogs

Create long advisor-client conversations grounded in each household profile, with evidence files for field-level traceability.

3

Extract and score

Run structured extraction from transcript text, match extracted records to dialog-grounded GT, and score field-level accuracy.

4

Validate holdout test

Evaluate a held-out test set that was not used while tuning the extractor or evaluator.

1. DATA GENERATION

Household data is scenario-driven and distributionally checked

The generator creates affluent US advisor-like households rather than generic population samples. Scenarios are explicit, weighted, and validated for coverage. Public-data anchors and curated priors drive income, assets, liabilities, household structure, risk tolerance, and lifecycle fields.

Coverage requirement

11 scenarios

Affluent conditioning

RIA-like population

Every household belongs to one named archetype. This avoids hidden rare cases and makes coverage auditable.

Income and wealth distributions are shifted upward and constrained by an affluent income floor.

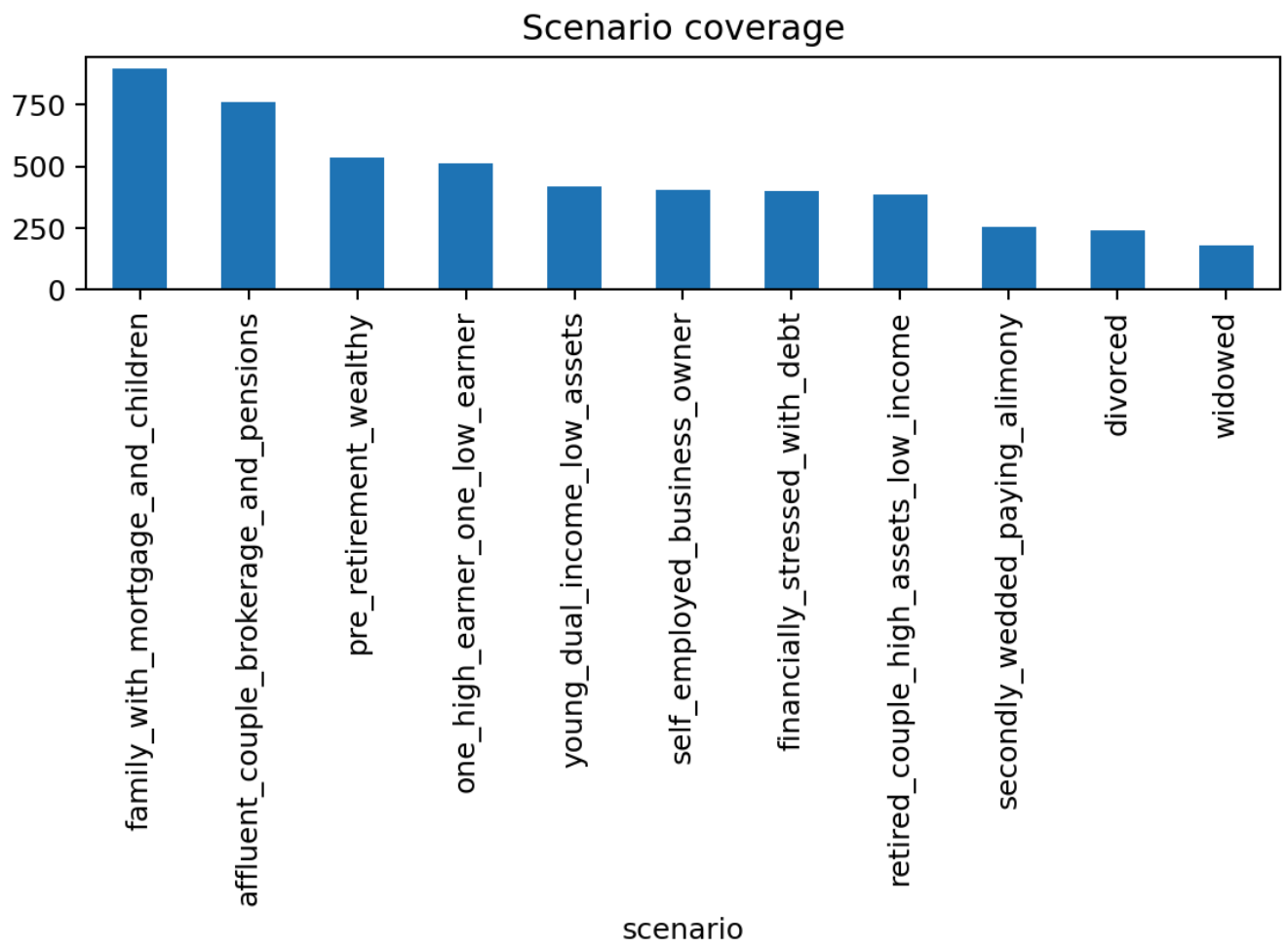
Validation

0 rule violations

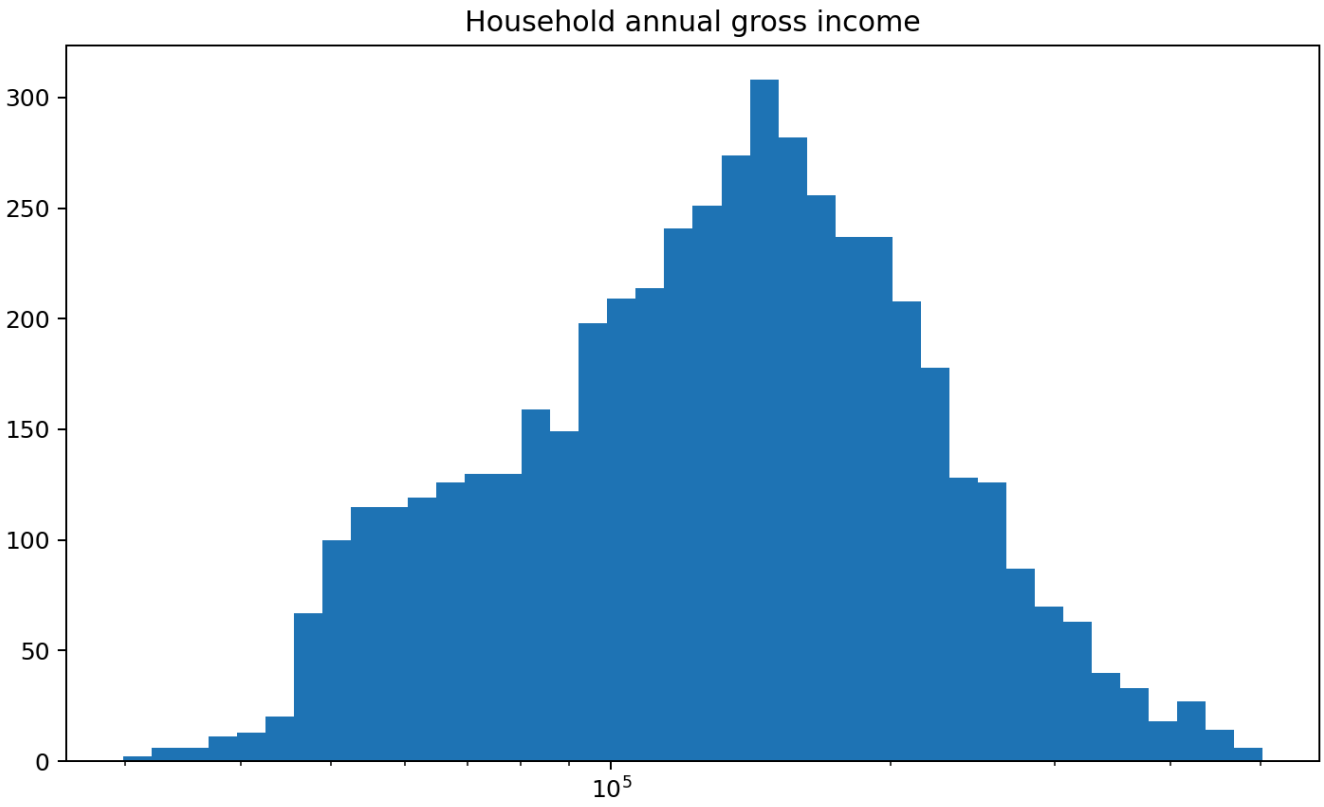
The generated dataset has no business-rule violations in the current artifact set.

Scenario coverage in the 5,000-household synthetic population

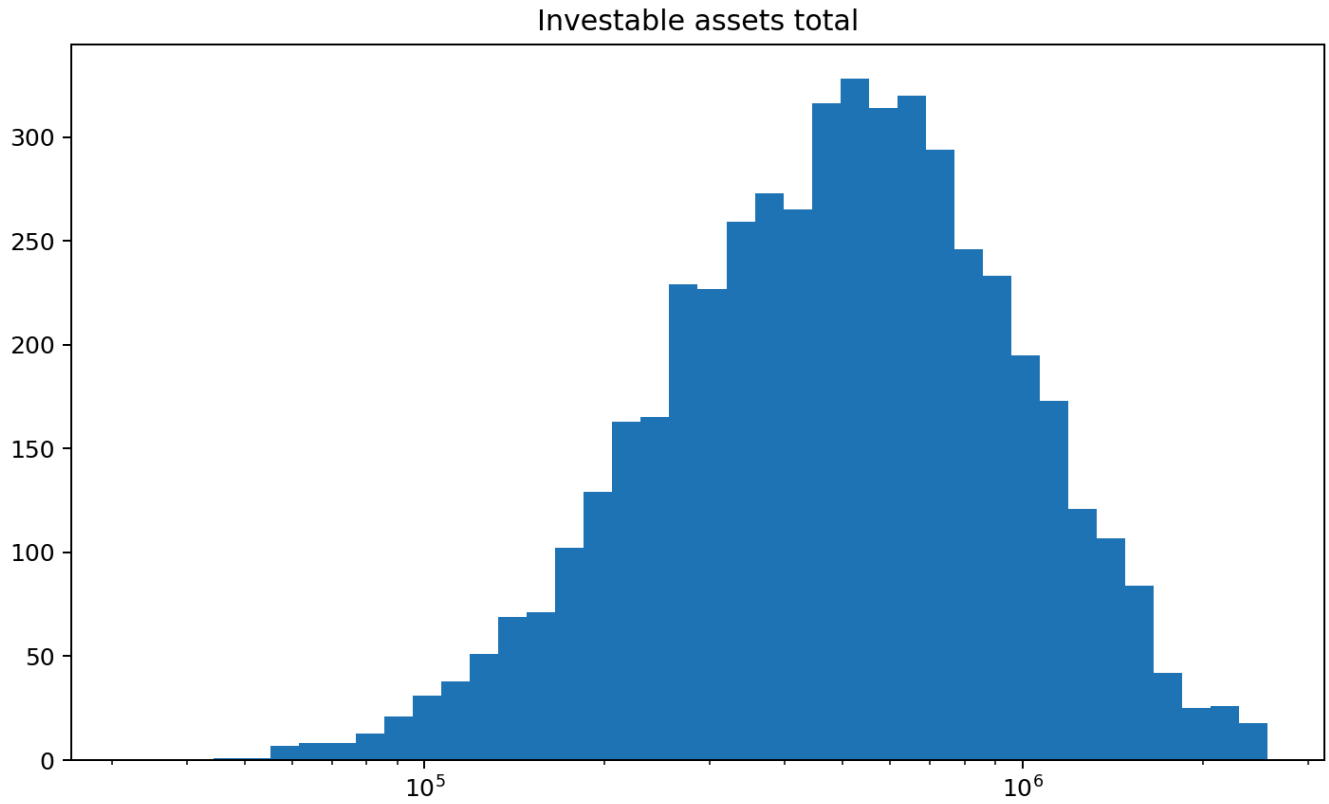
Scenario	Count	Share
family_with_mortgage_and_children	898	17.96%
affluent_couple_brokerage_and_pensions	761	15.22%
pre_retirement_wealthy	535	10.70%
one_high_earner_one_low_earner	512	10.24%
young_dual_income_low_assets	419	8.38%
self_employed_business_owner	406	8.12%
financially_stressed_with_debt	401	8.02%
retired_couple_high_assets_low_income	388	7.76%
secondly_wedded_paying_alimony	257	5.14%
divorced	243	4.86%
widowed	180	3.60%



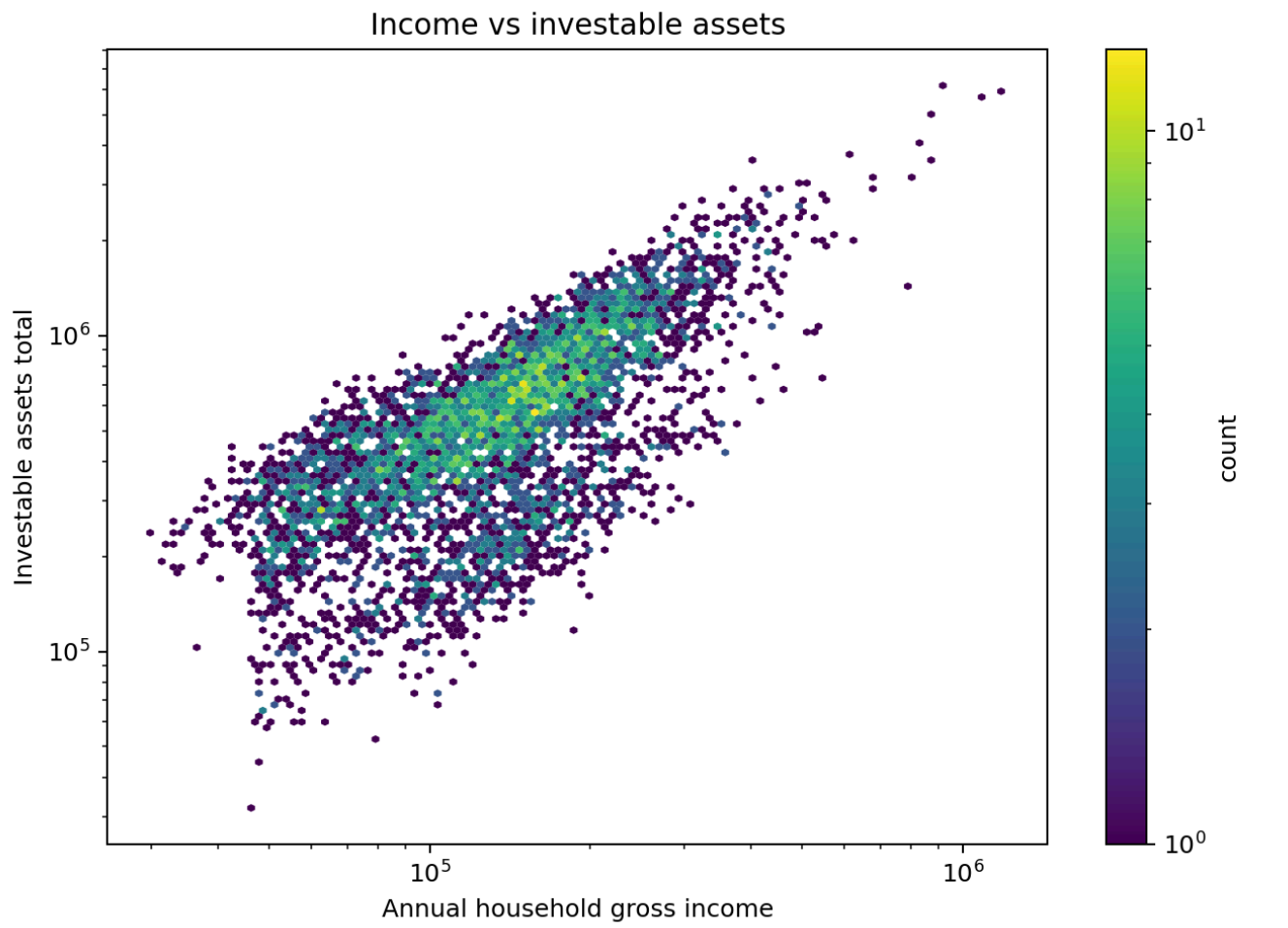
Scenario coverage confirms that all 11 target archetypes are represented.



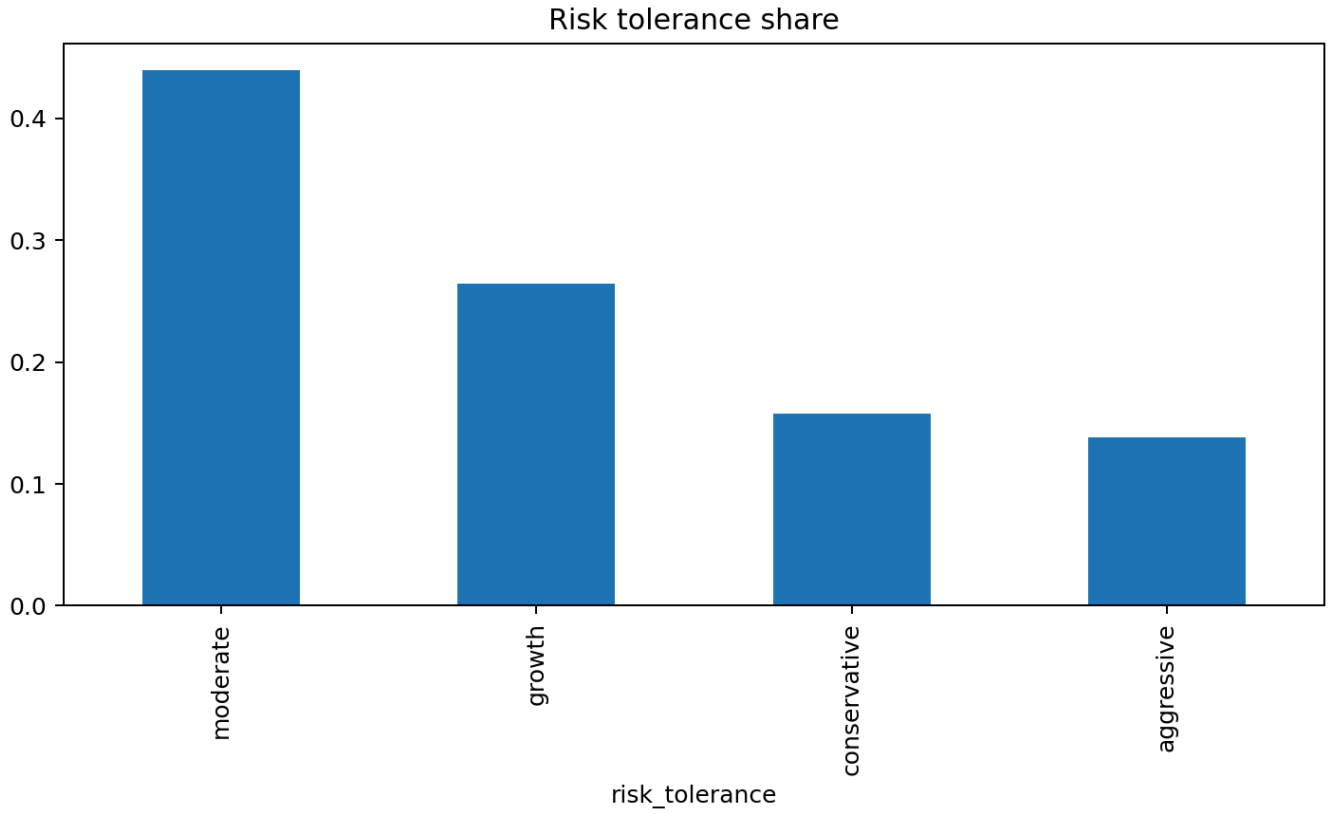
Annual household gross income distribution, conditioned toward affluent households.



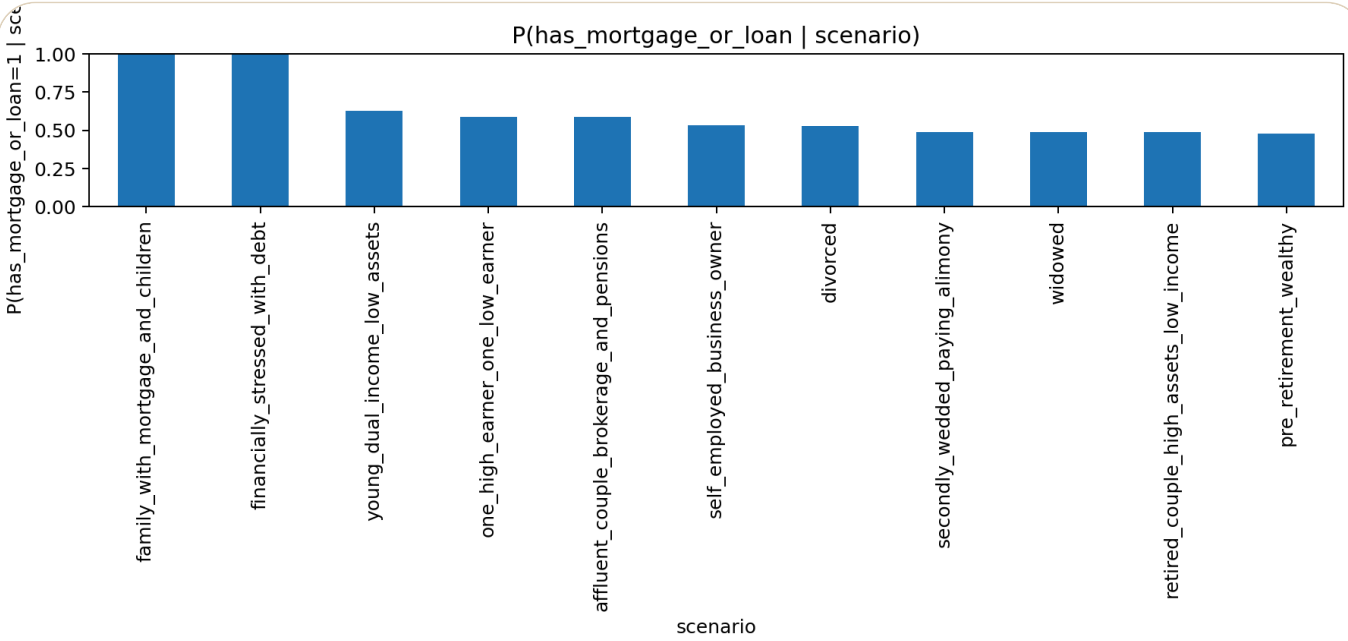
Investable assets distribution, generated as a noisy income-linked wealth model.



Income vs investable assets checks cross-field coherence and boundary artifacts.



Risk tolerance distribution is monitored as a categorical prior sanity check.



Conditional mortgage incidence by scenario checks dependence structure.

2. DIALOG GENERATION

Dialogs are generated in stages and checked for grounding and realism

Dialog generation does not ask a model to produce an entire transcript in one shot. It builds a persona, an outline, phase-level dialog turns, and state updates, then optionally finalizes the transcript into a more natural conversation.

Grounded transcript construction

The dialog generator consumes the household financial profile and scenario context. It writes a transcript, JSON metadata, evidence items, validation metrics, and plain-text transcript files.

Evidence artifacts map profile fields to advisor/client excerpts. These evidence files later become dialog-grounded GT for extraction scoring.

DeepSeek realism gate

A DeepSeek judge scores finalized transcripts on a 1 to 5 realism scale. The standard gate copies transcripts into a `realism_passed` subset when the realism score meets the configured threshold.

In the Train set, 316 dialogs are from the realism-passed subset. For the holdout test, the requested evaluation used 99 valid generated dialogs with judge artifacts retained.

Why this matters

The extraction benchmark is only meaningful if the dialogs sound like plausible financial-discovery conversations and if each scored field can be traced back to information present in the transcript. The realism gate helps with conversational quality; evidence-based grounded GT helps with fair scoring.

Dialog generation README | DeepSeek judge prompt | Extraction scoring details

3. MATCHING AND ERROR ACCOUNTING

Metrics are computed after record matching and grounded-aware field scoring

The evaluator compares extracted JSON to dialog-grounded GT. It normalizes both sides, pairs records by content when IDs are unstable, then scores fields cell-by-cell.

Record matching

Field scoring

Primary keys are used when reliable. Otherwise, content-based pairing is used for people, income lines, assets, liabilities, and protection policies.

Numeric fields use 1% relative tolerance. Multichoice fields are compared as sets. String-like fields are compared case-insensitively after normalization.

Error types

Errors are split into missing extracted values, invented extra values, value mismatches, and record-pairing failures.

Scoring assumptions

Assumption	Implementation	Why it matters
Dialog-grounded denominator	When `ground_truth_is_grounded=true`, only fields present in grounded GT enter the denominator.	Extractor is not penalized for facts never stated in the dialog.
ID exclusion	Primary keys and `_id` fields are excluded by default.	We evaluate semantic extraction, not synthetic key guessing.
Derived ratio exclusion	Fields ending in `_ratio` are excluded.	Ratio fields are derived and would otherwise double-count numeric facts.

Assumption	Implementation	Why it matters
PII/person-detail exclusions	Configured in `03_data_extraction/config/scoring_exclusions.json`.	Reduces noise from fields that are not central to financial extraction quality.

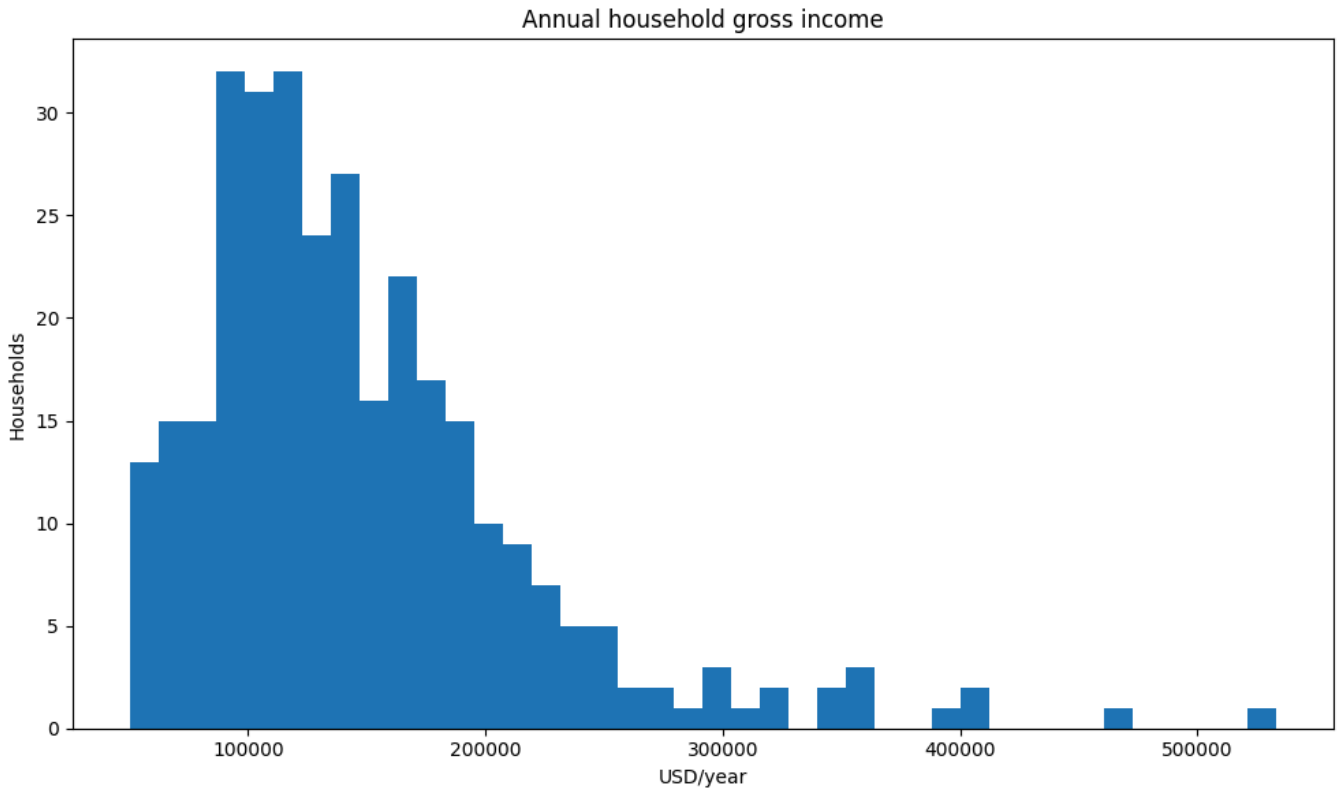
4. TRAIN AND HOLDOUT TEST LOGIC

Train was used for pipeline development; holdout test is a held-out evaluation

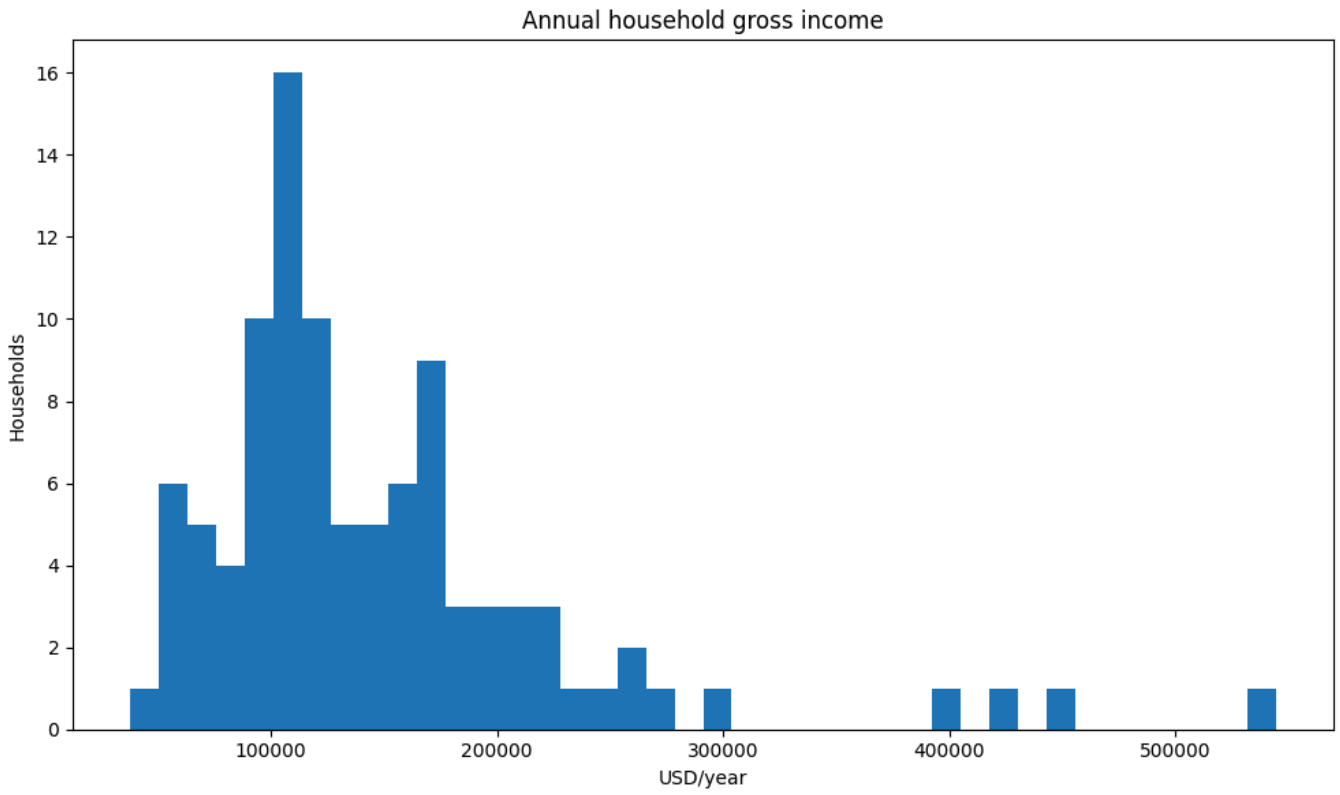
The 316 Train dialogs were used while debugging the extraction pipeline, prompts, scoring, and discrepancy analysis. The 99 holdout test dialogs were generated from households that did not overlap with the Train extracted households and were not used for extractor/evaluator tuning.

Split summary

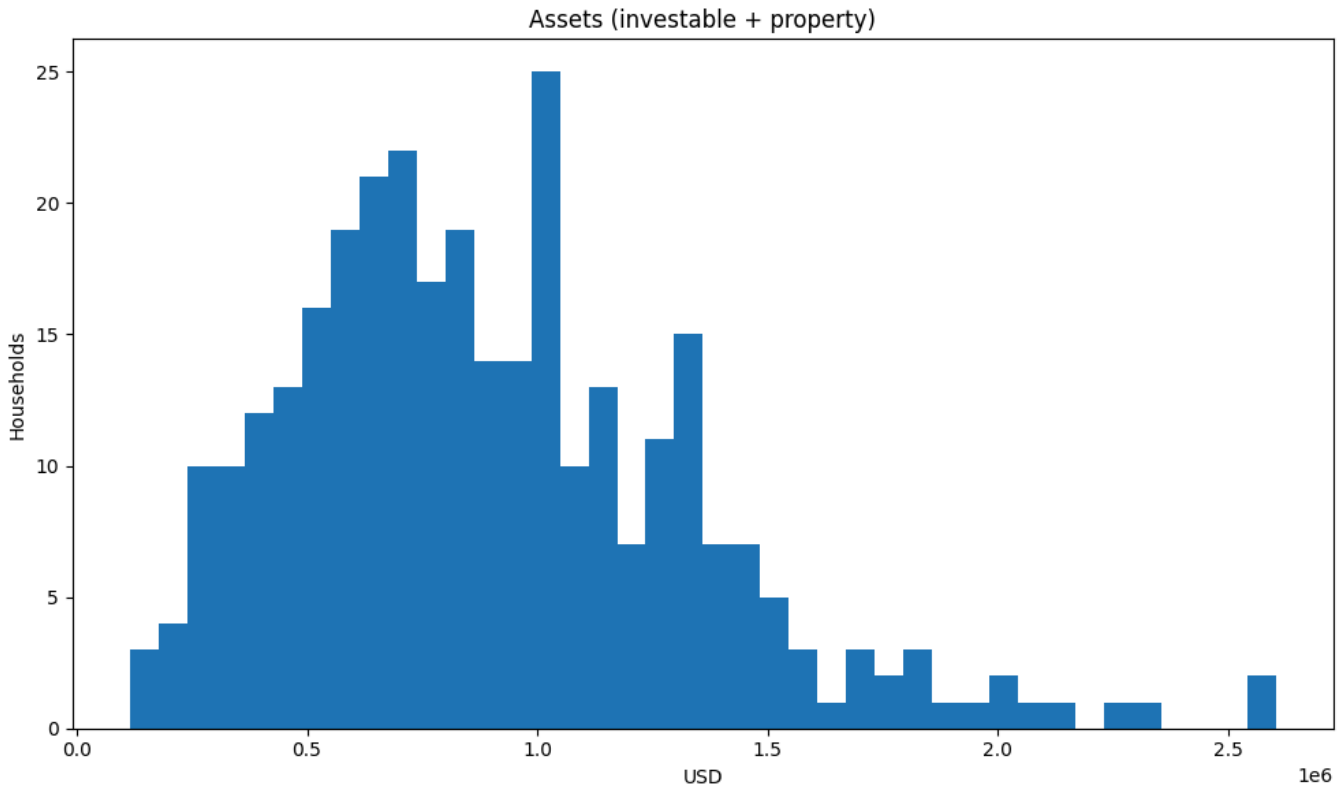
Split	Dialogs	Grounded GT pairs	Median income	P10/P90 income	Median assets	P10/P90 assets	Role
Train	316	316	$132,125	$78,300 / $226,850	$826,600	$389,600 / $1,444,400	Pipeline debugging and tuning
Holdout test	99	99	$121,900	$71,450 / $221,050	$821,650	$431,700 / $1,810,800	Held-out evaluation



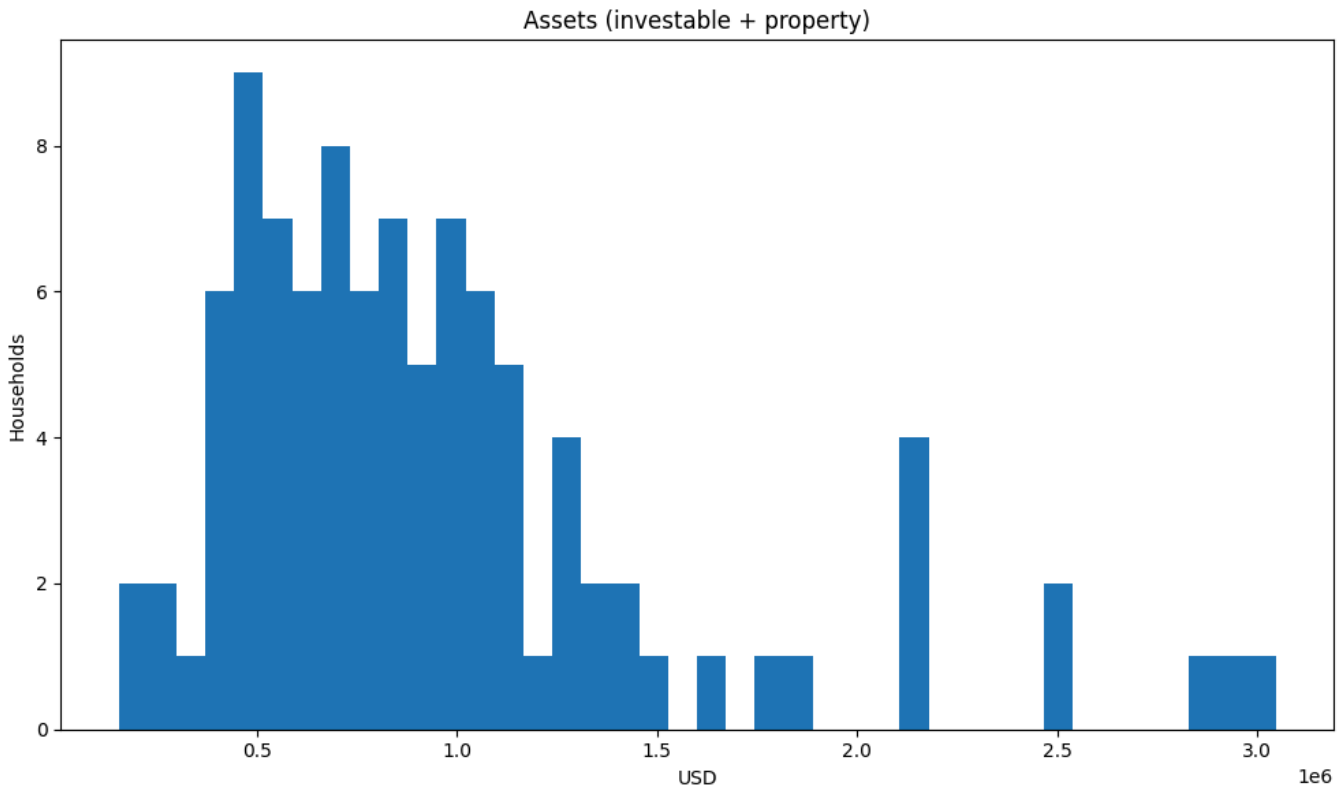
Train income distribution across 316 grounded dialogs.



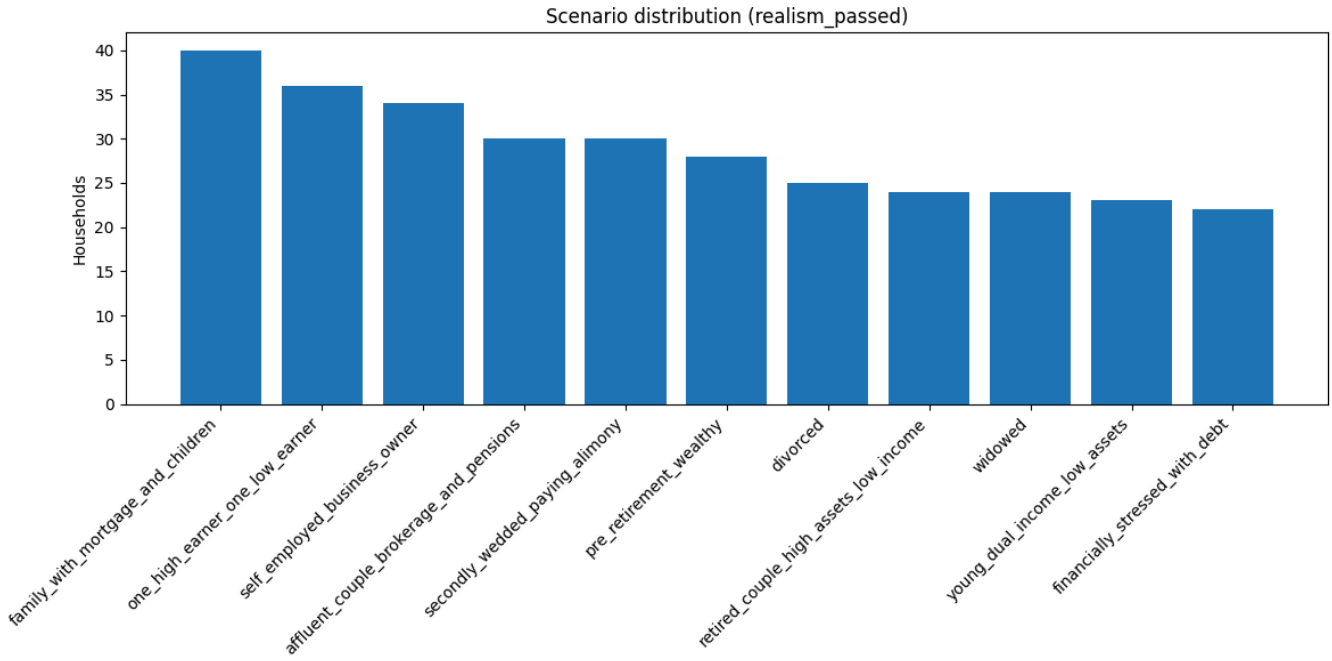
Holdout test income distribution across 99 held-out dialogs.



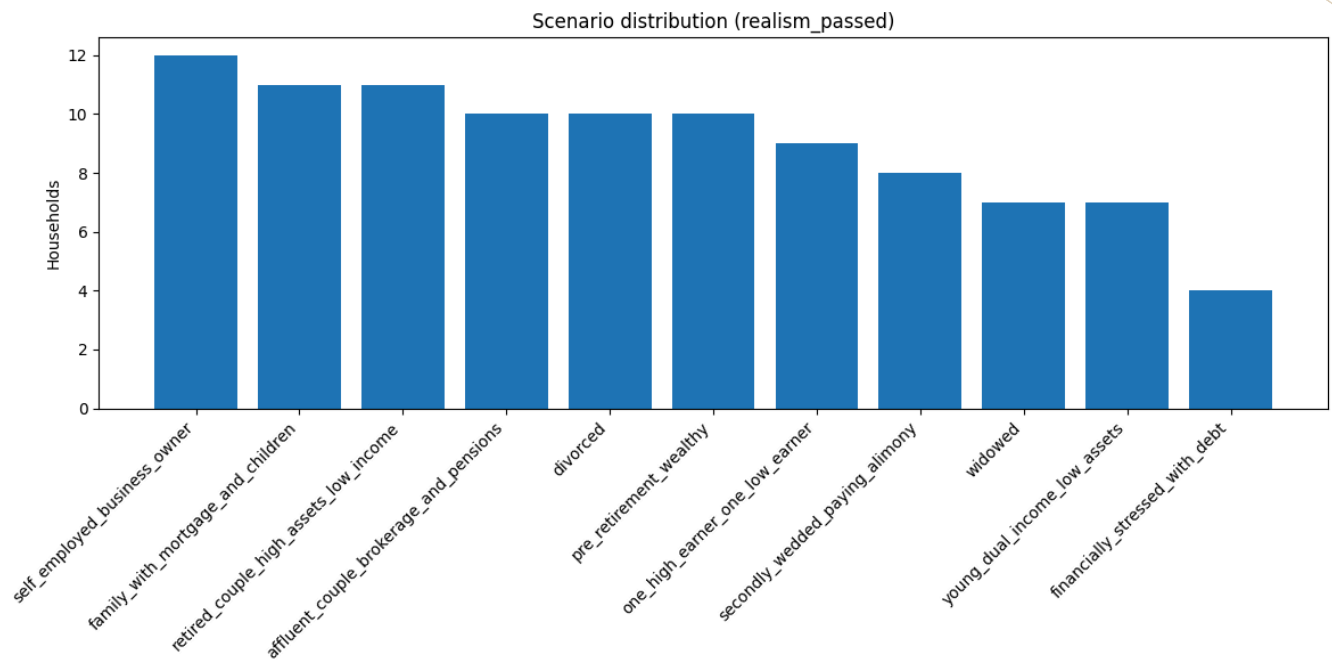
Train assets distribution across 316 grounded dialogs.



Holdout test assets distribution across 99 held-out dialogs.



Train scenario distribution. All target scenarios are represented.



Holdout test scenario distribution. The held-out set preserves broad scenario coverage.

5. RESULTS

Train and holdout test performance are closely aligned

Holdout test accuracy is nearly identical to Train at the aggregate field level. The strongest remaining errors are concentrated in a small number of asset ownership/type/provider cases, as shown in the

discrepancy reports.

Overall metrics

Metric	Train (316)	Holdout test (99)
Mean household field accuracy	**98.78%**	**98.74%**
Cell-level match rate	98.79%	98.74%
Dialogs with 100% correct fields	201 / 316 = 63.6%	58 / 99 = 58.6%
Dialogs with at least 95% correct fields	296 / 316 = 93.7%	95 / 99 = 96.0%
Dialogs with at least 90% correct fields	316 / 316 = 100.0%	99 / 99 = 100.0%
Missing extracted cells	73	33
Invented extra cells	0	0
Value mismatch cells	184	50

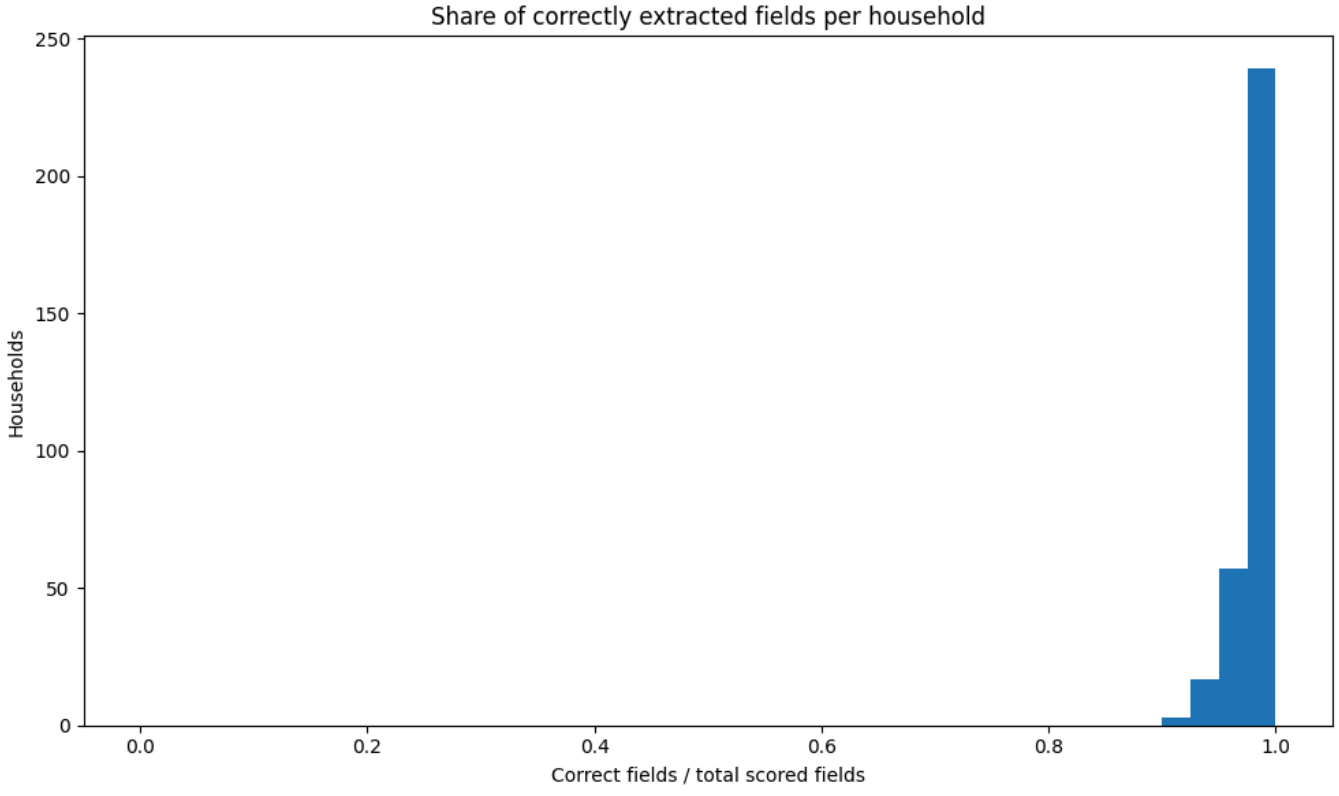
Train per-entity metrics

Entity	Missed	Errors	Invented	Cells
people	0.2%	0.5%	0.0%	2,602
liabilities	0.0%	0.0%	0.0%	1,338
protection_policies	0.0%	0.0%	0.0%	950
assets	0.8%	1.9%	0.0%	8,635
households	0.0%	0.1%	0.0%	5,056
income_lines	0.0%	0.1%	0.0%	2,648

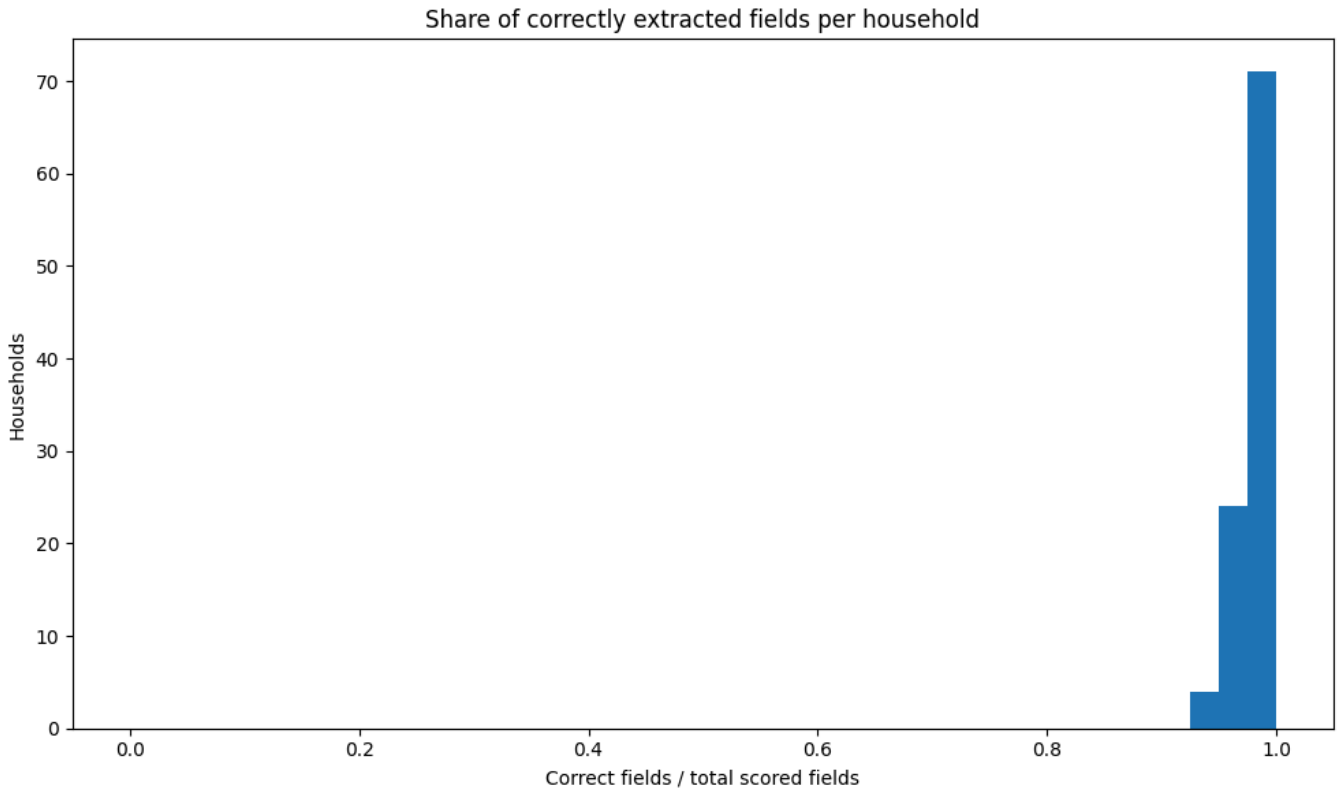
Holdout test per-entity metrics

Entity	Missed	Errors	Invented	Cells
people	0.0%	0.3%	0.0%	799
liabilities	0.0%	0.0%	0.0%	341

Entity	Missed	Errors	Invented	Cells
protection_policies	0.0%	0.0%	0.0%	270
assets	1.1%	1.7%	0.0%	2,713
households	0.0%	0.1%	0.0%	1,584
income_lines	0.4%	0.1%	0.0%	900



Train household-level extraction accuracy histogram for 316 dialogs.



Holdout test household-level extraction accuracy histogram for 99 dialogs.

6. QUALITY CONTROLS

Each stage has its own checks before the next stage consumes its outputs

Quality is controlled as a pipeline property, not only as a final accuracy number. We validate the generated household data, the dialog generation process, the dialog-grounded GT construction, extraction behavior on Train, and the full pipeline on the holdout test.

1. Synthetic data quality

The household generator is checked with business rules that enforce relationships between fields: lifecycle date constraints, parent/child age consistency, mortgage burden caps, alimony-marital-status compatibility, and future final-payment dates for non-revolving liabilities.

Statistical tests compare the generated data to priors using distributional diagnostics such as PSI and Jensen-Shannon divergence. Anomaly detection is run with two complementary models: an autoencoder reconstruction score and an IsolationForest score.

Rule violations | Distance to priors | Anomaly scores | Autoencoder top anomalies

IsolationForest top anomalies

2. Dialog generation quality

Dialogs are generated in pieces rather than as a single monolithic response: persona, outline, phase-by-phase transcript, state updates, and optional final polishing. This reduces drift and makes the conversation easier to validate incrementally.

Evidence artifacts map profile facts to transcript excerpts. The generation pipeline also writes metrics and uses validation checks, including regex-style and normalization-based checks, to catch fields that are not properly grounded in the transcript. A DeepSeek realism judge adds a separate conversational-quality gate.

Dialog QA details | Evidence prompt | DeepSeek realism prompt | Normalization rules

3. Ground truth quality

Extraction scoring uses dialog-grounded GT whenever evidence is available. This is a sparse GT profile built only from fields that have evidence in the transcript, so the evaluator does not ask the extractor to recover unspoken facts from the full synthetic profile.

The GT export reports how many dialogs were seen, how many profiles were written, how many evidence items were used, and whether any dialogs were missing profile metadata or household identifiers. During holdout evaluation, the pipeline confirmed 99 dialogs written, 99 grounded profiles used, and no missing profile/text rows.

Train grounded GT | Holdout grounded GT | Train pairing summary | Holdout pairing summary

4. Extraction quality on Train

The 316-dialog Train set was used to debug the extractor, prompts, normalization, rescue passes, record matching, and error reporting. This is where we iterated on the pipeline and inspected discrepancy tables to identify systematic extraction failures.

Train quality is monitored through extracted status reports, coercion issues, coverage aggregates, field-level discrepancy stats, value-mismatch tables, and accuracy histograms.

Train extraction report | Coercion issues | Field stats | Value mismatches

5. End-to-end pipeline quality on the holdout test

The holdout test validates the complete pipeline after tuning: held-out dialog generation, grounded GT export, extraction, record matching, scoring, discrepancy analysis, and final metrics. The holdout households do not overlap with the 316 Train households used during pipeline development.

This final gate is the closest approximation to an honest out-of-sample test: the same code and thresholds are applied, but the 99 dialogs were not part of extractor/evaluator iteration.

Holdout extraction report | Holdout accuracy report | Holdout discrepancy summary | Holdout discrepancy report | Holdout metrics table

7. NEXT STEPS

Research and product directions

The current benchmark gives us a controlled end-to-end baseline. The next phase should make the dialogs more realistic, stress the extractor under harder conditions, and connect extraction errors to real product outcomes.

1. Improve dialog realism with multi-model critique

Send generated dialogs to several model families such as Grok, Qwen, DeepSeek, GPT, Gemini, Claude, and others. Ask each model for a concrete action plan for improving realism, aggregate feedback across dialogs and models, and iteratively update the dialog-generation prompts.

2. Run noise-based stress tests

Inject controlled noise into dialogs and measure how extraction metrics degrade. Examples include typos, transcript artifacts, repeated turns, missing punctuation, speaker-label noise, irrelevant side conversations, partial truncation, and contradictory corrections.

3. Build adversarial dialog attacks

Train or prompt an adversarial model to make minimal dialog edits that break the extractor. Use these attacks to design defenses, stronger validation, or a more robust extraction architecture.

4. Compare structured output modes

Compare extraction with and without structured generation / structured output constraints. Measure not only accuracy, but also parse failures, coercion issues, latency, retry rate, and cost.

5. Study dialog length versus extraction quality

Quantify how transcript length affects extraction quality and latency. Identify the dialog size at which extraction should switch from single-pass processing to batching, chunking, or hierarchical extraction.

6. Add a pre-enrichment mode

Improve client experience by pre-filling as much information as possible from public or permissioned sources, such as public professional profiles or other available records. The dialog can then focus only on missing, ambiguous, or stale facts, shortening the conversation while still refreshing outdated information.

7. Make infrastructure failure-tolerant

Design graceful failover for API outages and provider degradation. For example, if GPT is unavailable, switch to Gemini or another provider, but only through a pipeline that has been explicitly tested and calibrated for that fallback model.

8. Evaluate multiple extractor models and ensembles

Test different LLMs as extractors and compare single-model, fallback, voting, and specialist-ensemble setups. Track accuracy, field-specific failure modes, latency, cost, and operational reliability.

9. Run real-world A/B validation

Compare this pipeline against the current production approach on real dialogs. Measure actual error patterns, human review burden, correction rates, and how extraction errors correlate with downstream product metrics.

ARTIFACT LINKS

Useful local references

These links point to files in this repository and should work when opening this HTML from the repo root.

Data generation

01 approach
01 README
Scenario coverage CSV
Scenario coverage plot

Dialogs

02 README
DeepSeek prompt
Train grounded GT
Holdout test registry

Extraction and results

03 approach
03 README
Train metrics
Holdout test metrics
Train discrepancy report
Holdout test discrepancy report

Generated from local artifacts in this repository. Open from the repository root so relative image and document links resolve correctly.

Back to top